// Day 3 of 10

Day 3

10 REST API Best Practices

Developers make these every day.
Don't be one of them.

→ Swipe through all 10

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #01

Use Proper HTTP Methods

GET should never modify data

```
// Bad
GET /deleteUser?id=5
GET /createOrder

// Good
GET    /users/5
POST   /orders
PUT    /users/5
DELETE /users/5
```

💡 HTTP method = intent, always follow it

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #02

Version Your API

Breaking changes break clients

```
// Bad
GET /api/users
-- No versioning at all!

// Good
GET /api/v1/users
GET /api/v2/users
-- Old clients still work
```

💡 Always start with /v1 from day 1

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #03

Return Proper Status Codes

200 for everything is wrong

```
// Bad
return 200 OK
{ "error": "User not found" }

// Good
404 Not Found
401 Unauthorized
422 Unprocessable Entity
201 Created
```

💡 Status codes tell the story

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #04

Use Nouns Not Verbs

The HTTP method IS the verb

```
// Bad
GET /getUsers
POST /createUser
DELETE /deleteUser/5

// Good
GET    /users
POST   /users
DELETE /users/5
```

💡 Resources = nouns, always

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #05

Always Paginate Lists

Returning 10k records = crash

```
// Bad
GET /users
-- returns all 50,000 users!

// Good
GET /users?page=1&size=20
{
  data: [ ... ],
  total: 50000,
  page: 1
}
```

💡 Default page size = 20 or less

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #06

Validate All Input

Never trust client data

```
// Bad
@PostMapping("/users")
public User create(@RequestBody User u){
  return repo.save(u); // no check!

// Good
@Valid @RequestBody UserDTO dto
// + @NotBlank @Email on DTO fields
// + handle MethodArgumentNotValid
```

💡 Validate at API boundary, always

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #07

Secure Your Endpoints

Public APIs are public targets

```
// Bad
// No auth check!
@GetMapping("/admin/users")
public List all(){ ... }

// Good
@PreAuthorize("hasRole('ADMIN')")
@GetMapping("/admin/users")
public List all(){ ... }
```

💡 Every endpoint needs auth check

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #08

Consistent Error Format

Random errors confuse clients

```
// Bad
"User not found"
{ error: 404 }
{ msg: "fail" }

// Good
{
  status: 404,
  error: "Not Found",
  message: "User id 5 not found",
  timestamp: "2024-01-01T10:00:00"
}
```

💡 One error format for the whole API

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #09

Don't Expose Sequential IDs

Sequential IDs reveal business data

```
// Bad
GET /users/1
GET /users/2
-- Easy to enumerate!

// Good
GET /users/a3f8c2d1-9b4e- ...
-- Use UUID or hashed IDs
```

💡 UUIDs hide your data volume

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #10

Rate Limit Your API

Without limits = easy to DDoS

```
// Bad
// No limits at all
// Bot sends 100k req/sec
// Server dies

// Good
@RateLimiter(name = "apiLimit",
  fallbackMethod = "rateLimitFallback")
// Return 429 Too Many Requests
```

💡 Rate limiting = basic production need

Shubham Bhati
linkedin.com/in/bhatishubham

```
done = true
```

Follow for more tips!

Like · Comment · Repost

Shubham Bhati
linkedin.com/in/bhatishubham